Gen AI

Role

- I have around **3+ years of experience in the IT industry** as a **Generative AI / Agentic AI Developer**, with hands-on skills in **Python, LangChain, Vector Databases (FAISS/Pinecone), OpenAI/Groq APIs, and RAG pipeline development**.
- I have handled **banking and enterprise-level projects**, where I built AI solutions to automate data extraction, summarization, customer query answering, and internal knowledge search.
- I worked extensively with **LLM APIs and open-source models** to fine tune, evaluate, and deploy custom AI applications as per business requirements.
- I have experience in designing **embedding pipelines**, chunking strategies, and building **RAG architectures** using framework like LangChain.
- I used Lang graph for multi agentic system recently.
- I have implemented **end-to-end GenAI solutions**, including:
  Tables → Document loaders → Embeddings → Vector DB → Retrieval → LLM generation → API deployment.
- Created **custom vector stores** to securely store client documents when external access was restricted due to compliance policies.
- Developed **automated ingestion pipelines** to read files from AWS S3 / shared drives, generate embeddings, and update vector databases continuously.
- Implemented **LLM based agents and workflows** to monitor data quality, summarize logs, validate migration outputs, and automate repetitive analytical tasks.
- Built **scheduled GenAI tasks** using cron / serverless functions to re-index documents, refresh embeddings, and maintain AI accuracy during data changes.
- Used **secure prompts, guardrails, and role-based access** to safely expose internal knowledge to partner applications and business users.
- We followed Agile Scrum methodology and used **JIRA** as the main ticketing tool.
- Used **GitHub** for code versioning and Jenkins for CI/CD pipeline integrations.
- That's a short introduction about me. Thanks!

Role

- Hi, my name is _____. I am working as a Gen AI and Agentic AI Engineer. I have experience in building end to end Gen AI solutions mainly using Python, LangChain and AWS services.
- In my current work, I am mainly building RAG based applications. I take documents from different sources like PDFs, APIs and databases, then I do preprocessing and chunking. After that I generate embeddings and store them in Pinecone. When user asks a question, I retrieve relevant chunks and pass it to LLM for generating accurate response.

- I have worked with AWS Bedrock, especially Nova Pro model, where I use it for text generation, summarization and Q&A use cases. I also use prompt engineering techniques like few-shot and role-based prompting to improve the output quality and reduce hallucination.
- For deployment, I build APIs using FastAPI and expose them through API Gateway. I use Lambda for serverless execution and EC2 when I need more control. I also handle logging, monitoring and error handling.
- In my project, I also worked on improving accuracy by tuning chunk size, using better embedding models and applying re-ranking techniques. I also handled duplicate data removal and ensured good response time.
- I use Jira for tracking tasks and working with team. I closely work with business team to understand requirements and convert them into AI solutions.

Role

- Hi, I'm_____. I have around 1+ years of experience, and currently I'm working as a Gen AI and Agentic AI Engineer in the _____ domain.
- In my recent project, I built an enterprise-level RAG solution for document analysis. We used GPT-4 class models for reasoning, OpenAI embedding models for vector generation, and stored embeddings in Pinecone for scalable search. In some modules, I also used FAISS for faster local similarity search.
- The RAG pipeline was built using LangChain, where we handled chunking, metadata filtering, and context injection properly. For preprocessing and orchestration, I used Python with Pandas and NumPy, and exposed the solution through REST APIs deployed on AWS EC2.
- I used langgraph for multi agentic system
- I used ragaas metric for evaluation.
- Recently I imporved retrieval accuracy. Initially, responses were slightly generic. So we optimized chunk size, overlap, and added better metadata filters, which improved answer relevance.
- We also used fast API, streamlit and git for version control and jenkins for the CICD,
- We follow agile scurm methodlogy with2 weeks of sprint

Day to day activities

- I first attend sprint meetings and discuss requirements, progress, and blockers with the team.
- I work on developing and enhancing RAG and Agentic AI applications for banking use cases.
- I create and optimize document ingestion pipelines, chunking strategies, embeddings, and retrieval logic to improve answer quality.
- I spend time evaluating LLM responses, analyzing retrieval accuracy, hallucination issues, and agent performance using test datasets.
- I also develop APIs, monitor application logs, investigate production issues, and implement fixes when needed.
- I collaborate with business users, QA teams, and architects to understand requirements and deliver new features.
- Along with development, I handle code reviews, Git commits, Jenkins deployments, and maintain project documentation in Jira.

Banking project

- The goal of my project was to help banking employees quickly find information from thousands of banking documents.
- Users could ask questions about loans, AML, compliance, payments, and internal banking policies using natural language.
- The system searched the relevant documents and retrieved the required information.
- Before generating a response, it validated the information against approved banking policies and compliance guidelines.
- It also supported policy search, document summarization, and policy comparison.
- This reduced manual effort, improved compliance, and helped employees make faster decisions.

Project architecture

- I first ingested banking documents from AWS S3 and internal repositories. For scanned documents, I used OCR to extract the text content.
- After extracting the content, I performed semantic chunking, generated embeddings, and stored the vectors in Pinecone along with metadata for efficient retrieval.
- When a user submitted a query, the LangGraph orchestrator analyzed the request and routed it to the respective agents.
- The retrieval agent fetched the most relevant document chunks from Pinecone, and the validation agent checked whether the retrieved information was reliable and relevant to the query.
- The compliance agent verified that the information aligned with banking policies and regulatory requirements before response generation.
- Finally, GPT-4 or Nova Pro generated the answer using the validated context. I used MCP to connect external systems and LangSmith to monitor, trace, and debug the complete workflow.

File validation

- During ingestion, when a file arrived in S3, I first read the file metadata using Python and validated the file name, extension, and size.
- For file names containing spaces or special characters, I used regular expressions to sanitize them by replacing spaces with underscores and removing unsupported characters.
- After validation, I stored the cleaned file name in metadata and continued with OCR, chunking, and embedding generation.
- This ensured consistent file handling and avoided retrieval or processing issues later in the pipeline.

Files from s3

- Documents were stored in AWS S3 and included PDFs, Word documents, text files, scanned documents, and images.
- I used boto3 to read files from S3 and identify the file type before processing.
- For text-based PDFs, I used PyPDF2 or pdfplumber to extract the content.
- For Word documents, I used python-docx, and for text files I used standard Python file handling.
- For scanned PDFs and image-based documents, I first converted PDF pages into images using pdf2image and then applied OCR using Tesseract or AWS Textract.
- After extraction, I separated text content from images and tables, cleaned the data, removed unwanted characters, and preserved metadata such as file name and page number.

- I then used RecursiveCharacterTextSplitter in LangChain to split the text into chunks, generated embeddings, and stored them in Pinecone for retrieval.

Libraries Used

- **boto3** → Read files from S3
- **PyPDF2 / pdfplumber** → PDF text extraction
- **python-docx** → Word documents
- **pdf2image** → Convert scanned PDFs into images
- **pytesseract** → OCR
- **AWS Textract** → OCR for forms, tables, and key-value pairs
- **Pillow (PIL)** → Image processing
- **LangChain Text Splitters** → Chunking

OCR

- OCR stands for Optical Character Recognition, and I use OCR when documents are scanned PDFs, images, or image-based files where normal text extraction does not work.
- In my project, I first tried extracting text using PDF libraries. If the document contained only images, I sent it through the OCR pipeline.
- For simple OCR use cases and POCs, I used Tesseract because it is open-source, runs locally, and works well for basic text extraction.
- However, Tesseract is less effective for complex layouts, tables, and forms, and its accuracy depends heavily on image quality.
- For production applications, I preferred AWS Textract because it provides better accuracy and can extract text, tables, forms, and key-value pairs.
- Textract integrates directly with S3 and scales well for large volumes of banking, healthcare, insurance, and KYC documents.
- After OCR extraction, I cleaned the text, performed chunking, generated embeddings, and stored them in Pinecone for retrieval in the RAG pipeline.

Generative AI

- Generative AI focuses on generating content such as text, images, code, audio, or videos.
- It takes an input prompt and generates an output.
- Examples include ChatGPT, Claude, Gemini, and image generation models.
- Most GenAI applications follow a request-response pattern.
- Examples: summarization, Q&A, translation, content generation, document extraction.

AI Agents

- An AI Agent is an LLM that can use tools and perform actions.
- Instead of only generating text, it can call APIs, query databases, search documents, or execute functions.
- Agents make decisions based on the user's request.

- They can interact with external systems before generating a response.

Agentic AI

- Agentic AI is a system of one or more AI agents working together to complete a goal.
- It involves planning, reasoning, tool usage, decision making, and multi-step execution.
- Multiple agents can collaborate, each handling a specific responsibility.
- Common patterns include Supervisor Agent, Retrieval Agent, Validation Agent, and Response Agent.
- Frameworks such as LangGraph and CrewAI are commonly used.

Tool Calling

- Tool Calling is the ability of an LLM to use external tools or functions to complete a user's request.
- Instead of relying only on its trained knowledge, the LLM can call APIs, databases, search systems, or Python functions to fetch real-time or external information.
- The LLM first decides whether a tool is needed. If required, it invokes the appropriate tool, receives the result, and then generates the final response.
- Tool calling makes LLM applications more accurate, dynamic, and capable of performing real-world tasks.

How Tool Calling Works

1. User asks a question.
2. LLM understands the request.
3. LLM decides whether it needs a tool.
4. Appropriate tool or API is called.
5. Tool returns the result.
6. LLM generates the final response using the tool output.

Multi- Agent System

- A multi-agent system is an AI architecture where multiple specialized agents work together to solve a task instead of using a single LLM.
- Each agent has a specific responsibility, such as retrieval, validation, compliance, or response generation.
- An orchestrator coordinates these agents and decides which agent should execute each step.
- This makes the system more modular, accurate, scalable, and easier to maintain.

Why Did We Use a Multi-Agent System

- The banking workflow involved multiple tasks that could not be handled efficiently by a single LLM.
- We needed separate validation, compliance, and retrieval before generating the final response.
- Using specialized agents improved response quality and reduced hallucinations.

How It Worked in My Project

Step 1 – User Query

- The user submits a banking-related question through the chatbot.

Step 2 – Orchestrator Agent

- The LangGraph Orchestrator receives the query.
- It analyzes the user's intent and decides which agents should participate.

Step 3 – Retrieval Agent

- The Retrieval Agent converts the query into an embedding.
- It searches Pinecone and retrieves the most relevant document chunks.

Step 4 – Validation Agent

- The Validation Agent checks whether the retrieved documents are relevant and have sufficient confidence.

Step 5 – Compliance Agent

- The Compliance Agent verifies that the retrieved information follows banking policies, KYC, AML, and regulatory guidelines.

Step 6 – Response Agent

- The Response Agent sends the validated context to GPT-4 or Amazon Nova Pro through Amazon Bedrock.
- The LLM generates the final response.

Step 7 – Return Response

- The final answer is sent back to the user through FastAPI and Streamlit.

My Role

- I Designed the LangGraph workflow.
- Built the Retrieval Agent using LangChain and Pinecone.
- Implemented Validation and Compliance checks.
- Integrated Amazon Bedrock for LLM inference.
- Used LangSmith to trace and debug the complete multi-agent execution.

Components of a Multi-Agent System

1. User Interface

This is where the user interacts with the application.

In my project, we used Streamlit as the frontend.

2. API Layer

Receives user requests and sends responses back to the frontend.

In my project, FastAPI handled all API requests.

3. Orchestrator (Supervisor Agent)

The brain of the multi-agent system.

Analyzes the user query and decides which agents should execute.

Coordinates communication between all agents.

In my project, this was implemented using LangGraph.

4. Specialized Agents

Each agent has a specific responsibility.

Retrieval Agent

Retrieves relevant document chunks from Pinecone.

Validation Agent

Checks the relevance and confidence of retrieved information.

Compliance Agent

Verifies banking policies, AML, and KYC compliance.

Response Agent

Calls the LLM and generates the final response.

5. Tools

Agents use tools to perform tasks.

Examples:

Pinecone Retriever

SQL Database

REST APIs

OCR (Textract)

Python Functions

6. Shared State / Memory

Stores intermediate results shared between agents.

Each agent can access the updated state without repeating work.

LangGraph manages this shared state.

7. Knowledge Source

Enterprise documents.

Vector Database (Pinecone).

SQL databases.

APIs.

SharePoint or S3.

8. LLM

Performs reasoning and generates responses.

Examples:

GPT-4

Claude Sonnet

Amazon Nova Pro

9. Monitoring

Tracks the execution of all agents.

In my project, I used LangSmith for tracing, debugging, token usage, and latency monitoring.

MCP

- MCP stands for Model Context Protocol.

- It is a standard protocol that allows LLMs and AI agents to communicate with external systems through a common interface.
- Instead of writing separate integrations for every database, API, or enterprise application, MCP provides a standard way for agents to discover and use external tools.
- It simplifies integration and makes AI applications more modular and scalable.

Why is MCP Needed

- It's a Standard way to connect external systems.
- Avoid writing custom integrations for every application.
- Easy integration with databases, APIs, SharePoint, Jira, etc.
- Makes multi-agent systems more scalable.
- Supports tool discovery and invocation.

Where Did I Use MCP?

In my banking project, the agents needed to access multiple enterprise systems such as document repositories, SQL databases, internal APIs, and knowledge sources. Instead of creating separate integrations for each system, we used MCP so that agents could access all these resources through a single standardized interface.

GEN AI Architecture

Gen AI is having Four main layers

- Data Layer
- Embedding Layer
- Retrieval layer
- LLM Layer

Data Layer

- Data layer is the source of the Information to the AI system
- Based on the information AI will understand the business.
- Say for Example Text documents, PDF, Databases, Websites, images, reports, emails etc..

Embedding Layer

- Embedding layer is used to convert the text to the numbers
- In this layer each of the document and data will be converted as vectors (numbers)
- These vectors are used to the get the relevant information quickly
- Encoding is also done in this layer

Retrieval Layer

Retrieval layer uses a **Vector Database** like FAISS, Pinecone, Chroma, Weaviate.

This layer is used get the semantic results,

When a user asks a question,
the system searches inside the vector DB and brings back the top-matching chunks.

This step is called **Retrieval**.

LLM Layer

- LLM Layer is the final layer it is used to generate the answer for the user query
- This is used to handle the
- User question &
- Retrieved relevant information
- Finally, LLM generates a **final answer** in natural language.
- Say for example LLM models are GPT, Llama, Claude, Groq, etc.)
- Decoding is also done in the LLM layer only.

Encoder

- The encoder splits text into tokens, converts them into numbers, and understands the meaning by looking at all words together.
- It gives outputs like classification or tagging, not sentence generation.
- I use encoder models for generating embeddings, semantic search, document retrieval in RAG, sentiment analysis, NER, and text classification. Since encoders can see the full context of a sentence, they provide better text understanding and representation.
- Example: BERT, RoBERTa, Sentence-BERT

Decoder

- The decoder converts the prompt into tokens and predicts the next word step by step.
  It generates new text instead of just understanding it.
- It takes the input prompt, breaks it into tokens, and predicts the next word one by one.
  It only looks at previous words, not future ones, to generate meaningful text.
  Finally, it produces new sentences like answers, stories, or code.
- I use decoder models for chatbot responses, summarization, question answering, code generation, and agentic AI workflows. Unlike encoder models that focus on understanding text, decoder models generate new text token by token based on the given prompt.

Example models: GPT , LLaMA , Gemma

Both encoder and decoder work on tokens.
Encoder uses tokens to understand text, while decoder uses tokens to generate text.

Encoder	Decoder
Understands the input text	Generates new text
Reads all words together	Predicts words one by one
Looks at past and future words	Looks only at past words

Encoder	Decoder
Gives meaning, label, or score	Gives sentences or paragraphs
Used in BERT	Used in GPT

Tokens

- Tokens are the smallest units of text that an LLM processes.
- The model cannot understand full sentences directly, so it first breaks the text into smaller pieces called tokens.
- A token can be a word, part of a word, punctuation mark, or special symbol.
  For example, "Banking systems are important" may become **["Bank", "ing", "systems", "are", "important"]**.
- After tokenization, each token is converted into a unique number called a Token ID, and the model works with these numbers.
- Tokens are the basic units of text processed by an LLM. Before the model can understand any sentence, the text is tokenized into words, sub-words, characters, or special tokens, converted into token IDs, and then processed by the Transformer model.

We have four types of tokens,

- Word tokens
- Sub-word Tokens
- Character Tokens
- Special Tokens

1. Word Tokens

- One token represents one complete word.
- Example: "I love banking" → ["I", "love", "banking"]
- Simple approach but struggles with new or rare words.

2. Sub-word Tokens (Most Common)

- Words are split into smaller meaningful parts.
- Example: "Banking" → ["Bank", "ing"]
- Handles new words, misspellings, and different word forms.
- Used by GPT, BERT, and LLaMA.

3. Character Tokens

- Each character becomes a token.
- Example: "Bank" → ["B", "a", "n", "k"]
- Very flexible but slower because token count becomes large.

4. Special Tokens

- Special control tokens used internally by the model.

Token	Purpose
[BOS]	Beginning of sentence
[EOS]	End of sentence
[PAD]	Padding shorter sequences
[CLS]	Classification tasks
[SEP]	Separate multiple sentences

Tokenization-

Tokenization is the process of splitting text into tokens before sending it to the model.

Example:

Text: "I don't like loans"

Tokens: ["I", "do", "n't", "like", "loan", "s"]

Popular Tokenization Methods

- BPE (Byte Pair Encoding) – Used in GPT models
- WordPiece – Used in BERT
- SentencePiece – Used in LLaMA and many modern LLMs

LLMs used Tokens for

- Reduce vocabulary size
- Handle new and unknown words
- Improve learning efficiency
- Convert text into numbers for processing
- Reduce memory and computation requirements

Why Tokens Matter

- Context Window: LLM can only process a limited number of tokens at a time.
- Cost: API pricing is usually based on token usage.
- Speed: More tokens mean higher processing time.
- Memory: More tokens consume more resources.

<u>Tokens in Prompting</u>

- When users send prompts to an LLM, the prompt text is converted into tokens.
- The model processes these tokens and generates output tokens.

- Both input tokens and output tokens are counted when calculating usage.

Tokens in RAG Systems

- In Retrieval-Augmented Generation systems, large documents are split into smaller chunks based on token size.
- For example, a PDF might be split into chunks of 500 tokens so that they fit within the LLM context window.

LLM	Context Window
GPT-4	8K – 32K
GPT-4 Turbo	128K
Claude 3.5 Sonnet	200K
Amazon Nova Pro	300K

LLM pricing is often **based on tokens**, not characters or words.

- More tokens → higher cost
- Input tokens + output tokens both counted

What are Vectors (Embeddings)

- Instead of comparing words directly, we convert sentence into numbers, and similar meaning sentences will have similar vector patterns.
- Embeddings means converting text into numerical vectors so that machine can understand similarity between two pieces of text.
- We used embeddings mainly for RAG system. We had lot of financial reports and compliance documents. So what we did was, after cleaning and chunking the documents, we generated embeddings for each chunk and stored them in FAISS initially.
- When user asks a question, we convert that question also into embedding and then do similarity search in vector database. The chunks which are closest in vector space are retrieved and sent to LLM as context.

Types of Embeddings

- **Word Embeddings** – Represent individual words. Examples: Word2Vec, GloVe, FastText.
- **Sentence Embeddings** – Represent entire sentences while preserving meaning. Examples: SBERT, all-MiniLM.
- **Document Embeddings** – Represent complete documents or paragraphs for document retrieval and RAG applications.
- **Text Embeddings** – General-purpose embeddings used in modern LLM applications. Examples: OpenAI text-embedding-3-small, Titan Embeddings.
- **Image Embeddings** – Convert images into vectors for image search and multimodal AI. Examples: CLIP embeddings.

- **Multimodal Embeddings** – Represent both text and images in the same vector space, enabling cross-modal search and retrieval.

Real openai

- In my project, we used embeddings to convert document chunks and user queries into vectors for semantic search.
- We mainly used **OpenAI text-embedding-3-small** because it provided high retrieval accuracy and better semantic understanding.
- When documents were uploaded, we performed chunking and generated embeddings for each chunk. These vectors were stored in the vector database.
- During query time, the user's question was also converted into an embedding, and similarity search was performed to retrieve the most relevant chunks.

Real titan

- I mainly used AWS Bedrock embedding models for enterprise use, because our system was fully on AWS and it gives better integration and security.
- I used models like Titan Embeddings for generating vectors. The main reason is it gives good semantic understanding, so even if user query is different from document wording, it still retrieves correct data.
- Compared to normal open-source models, this model is more stable and consistent in production. It also handles large scale data better and gives low latency.
- Another advantage is security and compliance, since data stays within AWS environment, which is important for enterprise clients.
- I also tested with some open-source embedding models, but Bedrock embeddings were giving better accuracy in retrieval and more reliable performance.

Open source SBERT

- For POCs and cost-sensitive projects, we used **SBERT**, an open-source model that runs locally.
- We used embeddings because they capture semantic meaning, improve retrieval quality, and help the LLM generate more accurate responses.

Evaluation of embedding

- I evaluated the embedding model by checking whether semantically similar documents were retrieved for a given query.
- I created a set of test questions and verified whether the expected documents appeared in the top search results.
- I measured retrieval metrics such as Recall@K and Hit Rate to understand retrieval performance.
- I compared different embedding models and selected the one that consistently returned the most relevant documents.
- I also reviewed retrieval results with business users to ensure the retrieved content matched their expectations.
- Based on the evaluation, I finalized the embedding model that provided the best retrieval quality for the application.

Why 1536

- 1536 is the default output dimension of Titan Embeddings G1 – Text model.
- Even though it has some drawbacks, I used 1536 because it was giving better retrieval accuracy in my project.
- Basically, in RAG pipeline, accuracy is more important than just saving memory. When I tested with lower dimensions using other models, the similarity search was not that strong, and sometimes it was missing correct context.
- With 1536, the embeddings were capturing more meaning from the text, so even if user question was different in wording, it was still retrieving correct chunks.
- Also, since I was using Titan Embeddings, 1536 is the default output, so I didn't have to do any extra transformation or compromise.
- From enterprise point of view, we also optimized cost in other ways like limiting chunk size, reducing duplicate data and tuning top_k, so overall system was still efficient.
- So I chose 1536 because it gave better balance of accuracy and performance for my use case.
- So when we use that model, it automatically generates vectors of size 1536, and we need to keep the same dimension in Pinecone for proper similarity search.
- Also, 1536 gives a good balance. It is not too small, so it captures enough meaning from the text, and not too large, so it keeps storage and query time under control.
- If dimension is too low, we may lose semantic information. If it is too high, it increases cost and latency without much benefit.

Why Not 1536

- One drawback of 1536 dimension is it increases storage size. Since each vector has 1536 values, when we store millions of records in Pinecone, it takes more memory and cost becomes high.
- It also impacts query latency. Higher dimension means more computation during similarity search, so response time can slightly increase compared to lower dimensions.
- Another point is over-representation. Sometimes not all 1536 dimensions are equally useful, so it may store some unnecessary information which does not add much value.
- It can also make scaling slightly harder, because index size grows faster and needs more resources.
- So overall, while 1536 gives good accuracy, the trade-off is higher cost, memory usage and slightly more processing time.

Challenges

- One challenge we faced was embedding model selection. Initially we used one embedding model, but retrieval quality was not very strong for financial terminology. Some domain-specific words were not matching properly. So we tested few embedding models and selected the one which gave better semantic similarity for finance content.
- For embedding model selection issue, what we did was create a small benchmark set of real analyst questions with expected document references. Then we tested multiple embedding models and compared retrieval accuracy manually. We checked which model was correctly matching financial terms like "exposure limit", "derivative contract", "regulatory capital" etc. Based on retrieval precision and similarity score consistency, we selected the model that performed best for finance content.

Ch|2

- Another issue we faced was when we upgraded embedding model version. New embeddings were not matching old stored vectors properly. Retrieval accuracy dropped. Then we understood we cannot mix embeddings from different models. So we re-generated all embeddings and version-controlled them properly.
- For embedding version upgrade issue, once we noticed retrieval accuracy dropped, we immediately stopped mixing old and new embeddings. We standardized on one embedding model version, re-generated embeddings for all documents, and rebuilt the full index. We also added embedding model version as metadata so future changes are controlled and tested before production.

Quality Drop in Embeddings

- Embedding quality drop basically means earlier similarity search was working properly, but now relevant results are not coming. So either wrong documents are retrieved or similarity scores are becoming weak.
- One main reason is changing the embedding model. If we switch to a new model but don't regenerate old vectors, then new query embeddings and old stored embeddings will not align properly. Similarity calculation becomes inconsistent. What I do in that case is regenerate all embeddings using the same model version and rebuild the index fully.
- Another reason is data change. If new type of content is added which is very different from earlier data, the existing embedding model may not capture semantics properly. For example, earlier text may be clean and structured, later it may be noisy or short-form text. In that case I review preprocessing, clean the text properly, and sometimes evaluate a better embedding model.
- Chunking also affects embedding quality. If chunk size is too big, embedding represents too many topics together. If too small, context meaning breaks. So when retrieval quality drops, I revisit chunk size and overlap strategy and regenerate embeddings again.
- Preprocessing issues are very common. If text contains a lot of repeated boilerplate, special characters, headers, or formatting noise, embeddings capture that noise. So I usually clean the text before embedding — remove unnecessary content, normalize spacing, remove irrelevant patterns.
- Index configuration can also cause quality drop. For example, wrong distance metric like using Euclidean instead of cosine similarity, or wrong index parameters. I verify similarity metric, rebuild index if needed, and test retrieval with known queries.
- Another reason is metadata filtering mistakes. Embeddings may be correct, but filtering logic may exclude relevant vectors. So I manually inspect top-k results without filters to see if embeddings are actually working.
- Also, if embedding model version changes silently in API provider, sometimes performance changes. So I track model versions and avoid automatic upgrades without evaluation.
- Whenever quality drops, my approach is simple. I test with fixed benchmark queries. I check what documents are retrieved. If retrieval is wrong, I focus on embeddings, chunking, preprocessing, and index. If retrieval is correct but final answer is wrong, then problem is in generation or prompt layer.

LLM

- LLM stands for Large Language Model.
- It is a deep learning model trained on massive amounts of text data to understand and generate human language.
- LLMs can perform tasks such as question answering, summarization, translation, content generation, code generation, and reasoning.
- They use Transformer architecture and attention mechanisms to understand context and relationships between words.
- Examples include GPT-4, Claude Sonnet, Amazon Nova Pro, and Llama.

Key components of LLM

1. **Tokens** – LLM reads tokens (small pieces of words) instead of full words.

Example: Artificial Intelligence → Art / ificial / Intelligence.

2. **Embeddings** – words are converted into numeric vectors so machines can process them.

Example: king → [numbers], queen → [numbers].

3. **Attention** – helps the model understand which words relate to each other.

Example: In the sentence 'The trophy didn't fit because it was too big', attention helps identify what 'it' refers to.

4. **Transformer architecture** – allows the model to analyze all words simultaneously and understand long context.

Real LLM

- In my project, mainly I used Nova Pro Model from AWS Bedrock. And most of the use cases like Q&A, summarization and content generation.
- Apart from that, I also tested with other models Like Claude in Bedrock to compare performance and accuracy, but Nova Pro was giving better results for our business use case.
- For embeddings, I used Bedrock embedding model Titan G1.

Real openai

- I used OpenAI's GPT-4 as the main LLM in my project for understanding user queries and generating responses.
- When a user submitted a question, I retrieved the most relevant document chunks from Pinecone and provided them as context to GPT-4.
- GPT-4 analyzed the user query along with the retrieved context and generated a clear, relevant answer for the user.

Real claude

- I used Claude 3.5 Sonnet as the main LLM in my project for understanding user queries and generating responses.
- When a user submitted a question, I retrieved the most relevant document chunks from Pinecone and provided them as context to Claude Sonnet.
- Claude Sonnet analyzed the user query along with the retrieved context and generated a clear, relevant answer for the user.

evaluate LLM

- I create a golden dataset containing questions and expected answers from the business domain for the llm evaluation
- Then I measure answer quality using metrics like Accuracy, Precision, Recall, F1-Score, Groundedness, Faithfulness, and Relevancy.
- For RAG systems, I also check Retrieval Accuracy, Recall@K, Hit Rate, Context Precision, and Context Recall.
- I validate hallucination by comparing the generated response with source documents and checking citation correctness.
- I use frameworks like RAGAS and LangSmith for automated evaluation and tracing.
- Finally, I perform human validation with business users to verify correctness, completeness, and usefulness of the response.

Golden Data

- First, I worked with business and SMEs to identify important questions which users will ask in real time. Then for each question, we manually prepared correct answers from trusted documents. This becomes our ground truth or golden dataset.
- After that, I created a Q&A pair dataset, where each question has expected answer and sometimes also the source document reference.
- Then I use this dataset to test my pipeline. I run queries through my system and compare the generated answer with golden answer.
- I also check retrieval quality, like whether correct chunks are coming or not. Based on this, I tune chunking, embeddings and prompts.
- In some cases, I also added edge cases and tricky questions to make sure system is robust.

Langsmith

- LangSmith is an observability, monitoring, debugging, and evaluation platform for LangChain and LangGraph applications.
- It helps track the complete execution flow of LLM applications.
- I used LangSmith to monitor agent execution, trace requests, debug failures, and evaluate response quality.
- It provides visibility into prompts, retrieved documents, LLM responses, token usage, latency, and errors.
- It is especially useful in multi-agent and RAG applications where multiple steps are involved.
- In my RAG project, I used LangSmith to trace the end-to-end flow from user query to retrieval and final response generation. If a user received an incorrect answer, I could inspect the retrieved chunks, prompts, and LLM output to identify whether the issue came from retrieval or generation.

Prompt Engineering

- Prompt engineering is basically how we design instructions for the model so that it gives the output we actually want.
- LLM is very powerful, but it only works based on what we give in the prompt. So the way we frame the question, the context we provide, the constraints we add… all that matters a lot.
- It is like guiding the model properly. If we give irrelevant instruction, we get irrelevant answer. If we give clear role, format, and limits, output will be accurate.

Project Prompt

- First, I use clear instruction based prompting. I clearly tell the model what to do, like answer only from given context, don't assume anything, and give short and precise answer.
- I also use role-based prompting. For example, I tell the model to act like a domain expert or support assistant, so it understands the tone and style.
- I use few-shot prompting in some cases, where I give 2 or 3 example questions and answers. This helps the model understand expected format.
- For RAG use case, I use context injection. I pass retrieved chunks along with the question and instruct the model to answer only from that context.
- I also define output format in prompt, like asking it to give bullet points or step-by-step answer, so response is structured.
- To reduce hallucination, I add instructions like "if answer is not in context, say I don't know".
- I also keep prompts simple and avoid unnecessary words, because long and complex prompts sometimes confuse the model.

Types

- Zero-shot prompting
- Few-shot prompting
- Chain-of-thought prompting
- Role-based prompting
- Instruction-based prompting
- Context-based prompting

Zero shot prompting means we just ask the question directly without giving examples. Like "Summarize this text in 3 lines." Model already has knowledge, so it tries to answer directly.

Few-shot prompting means we give 1 or 2 examples in the prompt and then ask it to do similar task. For example, if we want specific output format, we show one example input and expected output. This helps when we want consistency.

Chain-of-thought prompting means we ask the model to explain step by step reasoning. Like "Explain step by step." This improves logical tasks.

Role-based prompting means we assign a role like "You are a financial analyst" or "You are a medical assistant." That influences tone and context understanding.

Instruction-based prompting is when we clearly define output format, constraints, tone, length. For example, "Answer only from given context. If not found, say not available."

Context-based prompting is used in RAG systems where we pass retrieved documents as context and ask model to answer based only on that.

There is also prompt template design, where we dynamically insert variables like question, context, metadata into structured template. This is common in LangChain or LlamaIndex systems.

In my work, I usually combine role + context + strict output instruction. I don't make prompts too long. I clearly tell model what to do, what not to do, and how to format output. If output becomes too verbose, I restrict word limit. If hallucination happens, I add instruction to only answer from context.

Temperature in LLM

- Temperature is an LLM parameter that controls the randomness of the generated response.
- It influences how the model selects the next token during text generation.
- A low temperature makes the output more focused, predictable, and deterministic.
- A high temperature increases creativity and allows the model to generate more diverse responses.
- I usually use a lower temperature to get consistent and accurate answers.
- For content generation, brainstorming, or creative writing, a higher temperature can be used to generate more varied outputs.

Top P (Nucleus Sampling)

- Top-P controls how many likely words the model can choose from while generating a response.
- The model first ranks all possible words by probability.
- It then keeps adding the most likely words until a probability threshold is reached.
- The next word is selected only from that group.
- A lower Top-P gives more focused answers, while a higher Top-P allows more variety and creativity.
- In my projects, I usually use a moderate Top-P value to balance accuracy and response quality.
- For example, if Top-P is 0.8, the model keeps only the most likely words whose combined probability reaches 80% and chooses the next word from that set

Context Window

- The context window is the maximum number of tokens an LLM can process in a single request.

- It includes both the input provided to the model and the output generated by the model.
- The model can only remember and use information that is present within the current context window.
- If the token limit is exceeded, older parts of the conversation may be removed or summarized.
- A larger context window helps the model understand long conversations and process larger documents more effectively.
- In RAG applications, I manage context limits using chunking, retrieval, and conversation summarization so that only the most relevant information is sent to the LLM.
- For example, If a model supports 32K tokens, it can process up to approximately 32,000 input and output tokens combined in a single request.

Context Window of Popular LLMs

- **GPT-3.5** supports approximately **4,000 to 16,000 tokens**.
- **GPT-4** supports approximately **8,000 to 32,000 tokens** depending on the version.
- **GPT-4 Turbo** supports up to **128,000 tokens**.
- **Claude 3.5 Sonnet** supports up to **200,000 tokens**.
- **Amazon Nova Pro** supports up to **300,000 tokens**.
- **Llama 3** supports up to **128,000 tokens**, depending on the model version.

Transformer Architecture

- Transformer is the architecture behind modern LLMs like GPT, BERT, and Llama. It was introduced to overcome the limitations of RNNs and LSTMs, which process text sequentially and struggle with long-range dependencies.
- The flow starts with tokenization, where text is split into tokens. These tokens are converted into embeddings, and positional encoding is added so the model understands word order.
- The core component is Self-Attention, where each word looks at all other words in the sentence and identifies which words are important for understanding context. Internally, attention uses Query, Key, and Value vectors to calculate attention scores.
- Transformers use Multi-Head Attention, where multiple attention mechanisms run in parallel to capture different relationships such as grammar, context, and word dependencies. The output then passes through a Feed Forward Network for further processing.
- The original Transformer consists of an Encoder and a Decoder. The encoder understands the input, while the decoder generates the output. Models like BERT use only the encoder, GPT uses only the decoder, and T5 uses both.
- The biggest advantage of Transformers is parallel processing, better context understanding, and the ability to handle long sequences efficiently, which makes them the foundation of modern Generative AI systems

Self Attention

- Self-attention is the mechanism that helps the Transformer understand which words in a sentence are important to each other.
- Instead of looking at words one by one, every word can look at all the other words in the sentence and determine which ones provide useful context.

- For example, in the sentence 'The bank approved the loan because it met the criteria', the word 'it' refers to 'loan'.
- Self-attention helps the model identify that relationship. Internally, it uses Query, Key, and Value vectors to calculate attention scores and assign higher importance to relevant words.

Training vs Inference

- Training and inference are two different phases of an AI system. Training is the learning phase, while inference is the prediction phase.
- During training, the model learns patterns from large datasets by updating its weights based on the errors it makes.
- Training is computationally expensive and usually requires powerful GPUs or TPUs. It is costly and expensive, model.fit(X,y)
- During inference, the model uses its learned knowledge to generate answers or predictions without updating any weights.
- Inference is much faster than training and is what we use in production applications, model.predict(new_data)
- In my projects, I mainly worked with inference using pre-trained models like GPT-4, Claude Sonnet, and Nova Pro.
- For example, in a banking RAG application, when a user asks a question, the system retrieves relevant documents and the LLM generates the final response. That process is inference.

Hallucination in LLMs

- Hallucination occurs when an LLM generates incorrect or made-up information that is not present in the source data.
- Even though the answer may sound confident, it can be factually wrong or unsupported by the retrieved documents.
- This is one of the common challenges in Generative AI applications.
- The main types of hallucinations are factual hallucinations, source hallucinations, context hallucinations, reasoning hallucinations, and fabricated hallucinations.
- In RAG applications, I commonly encounter context and source hallucinations, which I reduce through retrieval validation, citations, and response verification.

How to reduce Hallucination

- I use RAG so the model answers from retrieved documents instead of relying only on its training data.
- I focus on retrieval quality because if the wrong document is retrieved, the final answer is likely to be wrong.
- I use clear prompts and instruct the model to answer only from the provided context.
- I keep the temperature low for banking and healthcare use cases to improve consistency.
- I include citations and validation checks so the response can be verified against the source documents.
- If the required information is not available, I prefer the model to say "I don't have enough information" rather than generate an answer.

Why Hallucination Happens

- LLMs are designed to predict the next token, so they can sometimes generate information that sounds correct but is actually wrong.
- If the model does not receive enough context, it may make assumptions and generate an incorrect answer.
- In RAG applications, poor retrieval is a common cause of hallucinations because the model receives irrelevant information.
- Ambiguous user questions can also lead the model to misunderstand the intent and produce incorrect responses.
- Conflicting or incomplete information in the source documents can confuse the model.
- Higher temperature settings can increase creativity but may also increase hallucinations.

Types of Hallucination

- Factual → Wrong facts
- Contextual → Not from given data
- Fabricated → Fake numbers/details
- Over-generalization → Extra info

Detection

- I first compare the generated response with the source documents or retrieved context.
- If the answer contains information that is not present in the source, I consider it a potential hallucination.
- I verify whether the citations actually support the generated response.
- I check the retrieval results to ensure the correct documents were retrieved.
- I use evaluation frameworks such as RAGAS and LangSmith to measure faithfulness and groundedness.
- For critical use cases, I also involve business users or SMEs to validate the answers.

Vector Database

- A vector database stores embeddings and helps retrieve semantically similar information.
- Unlike traditional databases that perform exact matching, vector databases perform meaning-based search.
- They are commonly used in RAG applications, chatbots, recommendation systems, and semantic search.
- The typical flow is: Document → Embedding → Vector Database → Retrieval → LLM → Response.

What did you use in your project

- In my project, I used **Pinecone** as the vector database.
- After chunking the documents, I generated embeddings and stored them in Pinecone along with metadata.
- When a user submitted a query, I converted the query into an embedding and retrieved the most relevant document chunks using similarity search.
- The retrieved content was then sent to the LLM to generate the final response.

Why Pinecone

- **It is fully managed and easy to scale in production.**
- **It supports metadata filtering, which helped us retrieve documents based on business criteria.**
- **It provided fast retrieval performance for our RAG application.**

FAISS

- I used FAISS during the POC and development phase to validate the RAG pipeline before moving to a production vector database.
- I generated embeddings for the document chunks and stored them locally in FAISS for similarity search.
- When a user submitted a query, I converted the query into an embedding and used FAISS to retrieve the most relevant chunks.
- Since FAISS runs locally, it was fast and cost-effective for testing and experimentation.
- Once the solution moved to production and required scalability, metadata filtering, and high availability, I migrated to Pinecone.

FAISS vs Pinecone

- FAISS is mainly used for local development and testing.
- Pinecone is used for production systems because it is managed and scalable.

OpenSearch

- I used OpenSearch when the application required both keyword search and semantic search.
- It helped users find information using exact terms as well as meaning-based queries.
- Documents were indexed in OpenSearch along with their embeddings.
- When a user submitted a query, OpenSearch performed hybrid search by combining keyword matching and vector similarity search.
- This improved retrieval accuracy, especially when users searched using specific banking terms, policy numbers, or document names.
- OpenSearch was also useful for monitoring, log analytics, and enterprise search applications.

Vector Similarity Metrics in Vector Databases and RAG Systems

- Similarity metrics are used by vector databases to measure how close a query vector is to stored vectors.
- They help retrieve the most relevant documents during semantic search.
- The three common metrics are Cosine Similarity, Euclidean Distance, and Dot Product.

Cosine Similarity

- Cosine Similarity measures how similar the direction of two vectors is.
- It focuses on semantic meaning rather than vector size.
- Higher score means higher similarity.

- It is the most commonly used metric in NLP and RAG applications.
- In my project, I used Cosine Similarity with Pinecone for document retrieval.

Euclidean Distance

- Euclidean Distance measures the physical distance between two vectors.
- Smaller distance means the vectors are more similar.
- It is commonly used in clustering and machine learning applications.
- It is less preferred for semantic text retrieval because it is sensitive to vector magnitude.

Dot Product

- Dot Product measures similarity using both vector direction and magnitude.
- Higher value generally means higher similarity.
- It is computationally efficient and commonly used in high-performance retrieval systems.
- Some embedding models are optimized specifically for dot-product search.

Why did you choose Cosine Similarity

- I chose Cosine Similarity because my use case involved text embeddings and semantic search.
- I wanted to retrieve documents based on meaning rather than exact distance or vector magnitude.

What is ANN

- ANN stands for Approximate Nearest Neighbor.
- Recent Vector databases use ANN algorithms to quickly find similar vectors without comparing every vector in the database.
- This improves retrieval speed, scalability, and latency.

HNSW-

- HNSW stands for Hierarchical Navigable Small World.
- It is an ANN (Approximate Nearest Neighbor) algorithm used for fast vector similarity search.
- Instead of comparing a query with every vector, HNSW organizes vectors into a graph structure.
- The algorithm navigates through the graph to quickly find the nearest vectors.
- This reduces search time while maintaining high retrieval accuracy.
- Many modern vector databases use HNSW internally for semantic search and RAG applications.

Why is HNSW used

- Fast retrieval
- Low latency
- High scalability
- Works well with millions of vectors
- Good balance between speed and accuracy

Almost all modern vector databases use ANN (Approximate Nearest Neighbor) algorithms internally for fast retrieval.

- **Pinecone** → Uses ANN internally for scalable similarity search.
- **FAISS** → Uses ANN indexes such as IVF and HNSW.
- **OpenSearch** → Uses HNSW-based ANN search.
- **Qdrant** → Uses HNSW ANN indexing.
- **Milvus** → Supports multiple ANN algorithms including HNSW and IVF.
- **Weaviate** → Uses HNSW for ANN search.

Comparison Between the Three Metrics

- The three common similarity metrics are Cosine Similarity, Euclidean Distance, and Dot Product.
- Cosine Similarity is mainly used in RAG and NLP because it focuses on semantic meaning. Euclidean Distance measures the actual distance between vectors and is commonly used in clustering.
- Dot Product considers both magnitude and direction and is often used in high-performance retrieval systems.
- In my project, I used Cosine Similarity with Pinecone for semantic document retrieval

Metric	What it Measures	Best Use Case	Result
Cosine Similarity	Direction of vectors	RAG, NLP, Semantic Search	Higher is better
Euclidean Distance	Physical distance between vectors	Clustering, ML	Lower is better
Dot Product	Direction + Magnitude	Fast Retrieval Systems	Higher is better

How is the Similarity Metric Decided?

- The similarity metric depends on the embedding model and the use case.
- For NLP and RAG applications, Cosine Similarity is usually preferred because it captures semantic meaning effectively.
- For recommendation systems and high-speed retrieval applications, Dot Product is often used because it is computationally efficient.
- For clustering and spatial analysis problems, Euclidean Distance is commonly used because it measures actual distance between vectors.
- The selected metric must also be supported by the vector database and embedding model being used.
- In Pinecone, the similarity metric is configured while creating the index.

When creating a vector index in Pinecone, the metric is configured manually.

Example:
metric='cosine'
metric='dotproduct'
metric='euclidean'

FAISS vs Pinecone

Feature	FAISS	Pinecone
Type	Library	Managed Vector Database
Hosting	Self-hosted	Cloud-managed
Metrics	L2 / Inner Product	Cosine / Dot Product / Euclidean
Scalability	Limited by infrastructure	Highly scalable
Best Use	Local and research projects	Production RAG systems

Metadata Filter in vector DB

- Metadata filters are used to narrow down the search results before retrieval.
- Along with embeddings, I store metadata such as document type, department, page number, source, date, and user role in Pinecone.
- When a user submits a query, the retriever passes the metadata filter to Pinecone.
- Pinecone searches only within the documents that match the filter criteria.
- This improves retrieval accuracy and reduces irrelevant results.
- In my project, I commonly used metadata filters such as document type, business category, and department during retrieval.

Chunking

- Chunking means breaking large documents into smaller pieces so that LLMs can process them efficiently. It is mainly used before creating embeddings in a RAG pipeline.
- Chunking means breaking large documents into smaller meaningful pieces before generating embeddings.
- If chunking is wrong, numbers or clauses get split, and LLM gives incomplete answer.

Types of Chunking

1. Fixed-size chunking
2. Sentence-based chunking

3. Paragraph-based chunking
4. Overlap chunking
5. Semantic Chunking

Chunk Overlap

Chunk overlap means repeating some part of previous chunk in the next chunk to preserve context. Typical: chunk size = 500–1000 chars, overlap = 100–200 chars.

Size chunking. In this we split document into equal size chunks, like 500 tokens each. It is simple and fast, but problem is it may break meaning in between sentences.

Second is sentence based chunking. Here we split based on sentences using NLP. So each chunk contains full sentences. This is better than fixed size because meaning is preserved, but still it may not group related content properly.

Third is paragraph based chunking. In this we split based on paragraphs. This keeps better context, but sometimes paragraphs can be too long and may exceed token limit.

Fourth is overlap chunking. Here we add some common content between chunks. For example, last 50 tokens of previous chunk will be included in next chunk. This helps to maintain continuity of context.

Fifth is semantic chunking. This is more advanced. Here we group sentences based on meaning similarity. So related sentences are kept together in same chunk. This gives better quality for retrieval.

Project Chunking

- We used both semantic chunking and overlap chunking together to improve retrieval accuracy.
- First, we split the document into sentences using NLP. Then I generate embeddings for each sentence. After that, I check similarity between consecutive sentences. If similarity is high, I group them into one chunk. If similarity drops, I start a new chunk. This way chunks are formed based on meaning, not just size.
- Once semantic chunks are ready, I apply overlap on top of that. For example, I take last few sentences from previous chunk and add it to next chunk. This helps in maintaining continuity of context.
- So overall flow is like, first semantic grouping based on similarity, then apply small overlap between chunks.
- This approach helped me in two ways. One is better retrieval accuracy because chunks are meaningful. Second is better context understanding for LLM because important information is not missed between chunks.

- Why I used this is because semantic chunking ensures that each chunk has meaningful context. And overlap chunking ensures no important information is lost between chunks.
- Compared to fixed or paragraph chunking, this approach gives better retrieval accuracy. Because when user asks question, Pinecone returns more relevant chunks, not broken or incomplete data.

Why Chunking is Needed

- LLMs have context window limits, so large documents cannot be sent to the model at once.
- Chunking breaks large documents into smaller, meaningful sections that can be processed efficiently.
- It improves retrieval accuracy because the system retrieves only the relevant part of the document instead of the entire document.
- It reduces token usage and overall LLM cost.
- It helps provide more focused context to the model, which reduces the chances of hallucinations.

Challenges Chunk

First issue was context break. Means important sentence or number was getting split between two chunks. So when retrieval happened, LLM was seeing only half information. In finance domain that caused incomplete answers. To fix this, we introduced overlapping chunks. We kept small overlap between consecutive chunks so context continuity was maintained.

Second challenge was chunk size tuning. If chunk was too small, retrieval was precise but missing broader explanation. If chunk was too large, irrelevant content got passed to LLM and latency increased. To solve this, we tested multiple chunk sizes using real analyst questions. We measured retrieval accuracy and response time, then selected balanced size that gave good relevance without increasing token cost.

Third challenge was different document formats. Some documents had proper headings and sections, others were raw text. One fixed logic was not working for all. So we implemented section-based chunking for structured reports, where we first split by headings, then applied token limits. For unstructured text, we used token based splitting with overlap.

Fourth challenge was duplication due to overlap. Because of overlap, similar content was getting retrieved multiple times, increasing token usage. To overcome that, we added deduplication logic before sending chunks to LLM. We filtered highly similar chunks using similarity comparison.

RAG

RAG (Retrieval-Augmented Generation) = Search + LLM
It improves accuracy by using external data instead of guessing.

RAG Architecture

- First, documents come from sources like PDFs, S3, APIs, databases, and shared drives.
- Then I perform preprocessing, OCR if needed, and semantic chunking with overlap.
- After that, I generate embeddings using models like Titan Embeddings or OpenAI embeddings and store them in Pinecone.
- When a user asks a question, the query is converted into an embedding and similarity search is performed in the vector database to retrieve top relevant chunks.
- Those retrieved chunks along with the user query are passed to the LLM such as GPT-4 or Nova Pro through a prompt template.
- Finally, the LLM generates a grounded response, and I also apply validation, citation checking, and guardrails to reduce hallucination and improve accuracy.

Components of a RAG System

The main components of a RAG system are the data source, chunking component, embedding model, vector database, retriever, prompt builder, and LLM. Documents are chunked and converted into embeddings, stored in a vector database, retrieved based on the user query, and then passed to the LLM to generate the final response.

- **Data Source** – PDFs, Word documents, databases, emails, websites, APIs, etc.
- **Chunking Component** – Splits large documents into smaller chunks for efficient retrieval.
- **Embedding Model** – Converts document chunks and user queries into vector representations.
- **Vector Database** – Stores embeddings and performs similarity search. Examples: Pinecone, FAISS.
- **Retriever** – Retrieves the most relevant chunks from the vector database based on the user query.
- **Prompt Builder** – Combines the retrieved context with the user's question.
- **LLM** – Generates the final response using the provided context. Examples: GPT-4, Claude 3.5 Sonnet, Amazon Nova Pro.
- **Evaluation & Monitoring** – Tools such as LangSmith and RAGAS are used to evaluate and monitor performance.

Evalute RAG

- I evaluate RAG in two ways: offline evaluation before deployment and online evaluation after deployment.
- In offline evaluation, I create a golden dataset with user questions, expected answers, and source documents.
- Then I measure metrics like Recall@K, Hit Rate, Context Precision@k, Context Recall, Faithfulness, Answer Relevancy, and Groundedness using frameworks like RAGAS.
- This helps me verify whether retrieval and generation are working correctly before production.
- In online evaluation, I monitor live user interactions and track metrics such as response latency, user feedback, citation accuracy, hallucination rate, answer acceptance rate, and escalation rate.

- I also use LangSmith dashboards and logs to identify retrieval failures, prompt issues, or poor-quality responses.
- By combining offline quality metrics and online production monitoring, I can continuously improve the RAG system accuracy and user experience.

When RAG may fail

- RAG can fail mainly when the retrieval part is not working properly.
- For example, if the documents are outdated, chunking is not done correctly, embeddings are not capturing the meaning properly, or the retriever is not fetching the right chunks, then the LLM will not get the correct context.
- I have also seen issues with scanned PDFs where OCR extraction was poor, which affected retrieval quality.
- Sometimes Top-K is too low, important chunks are missed, or the prompt is not designed properly.
- In most cases, when RAG fails, the root cause is usually retrieval quality rather than the LLM itself.

Retriever Concept in RAG

- Retriever is a component that fetches the most relevant document chunks from the vector database based on the user query.
- It acts as the bridge between the Vector Store and the LLM.
- Instead of sending all documents to the LLM, only the most relevant chunks are retrieved.
- This improves answer accuracy, reduces token usage, and helps reduce hallucinations.
- The retrieved chunks are then passed to the LLM as context for response generation.

How it Works

1. User submits a query.
2. Query is converted into an embedding.
3. Retriever searches the vector database.
4. Similarity search is performed using cosine similarity, dot product, or Euclidean distance.
5. Top-K relevant chunks are retrieved.
6. Retrieved chunks are sent to the LLM.
7. LLM generates the final response.

Types of Retrievers

- **Similarity Retriever** → Retrieves most similar chunks.
- **MMR Retriever** → Retrieves relevant and diverse chunks.
- **Hybrid Retriever** → Combines keyword and vector search.
- **Metadata Retriever** → Filters based on metadata such as document type, department, or date.

Top K

- Top-K is a sampling technique used in LLMs to control how the next token is selected.
- The model first ranks all possible next words based on probability.
- Instead of considering all words, it keeps only the top K most probable words.
- The next word is selected from those K words.
- A smaller K produces more focused and predictable responses.
- A larger K allows more diversity and creativity in the output.
- I used Top-K = 3 to keep the response focused on the most likely words.
- Since it was a banking application, accuracy was more important than creativity.
- A smaller Top-K helped generate more consistent and predictable responses.

Top P (Nucleus Sampling)

- Top-P controls how many likely words the model can choose from while generating a response.
- The model first ranks all possible words by probability.
- It then keeps adding the most likely words until a probability threshold is reached.
- The next word is selected only from that group.
- A lower Top-P gives more focused answers, while a higher Top-P allows more variety and creativity.
- In my projects, I usually use a moderate Top-P value to balance accuracy and response quality.
- For example, if Top-P is 0.8, the model keeps only the most likely words whose combined probability reaches 80% and chooses the next word from that set

Search Type in Retriever

- Search Type controls how the Retriever fetches chunks from the Vector Database.
- It determines the retrieval strategy before sending context to the LLM.
- Different search types are used based on the use case and retrieval requirements.

1. Similarity Search

- Retrieves the chunks with the highest similarity score.
- Uses cosine similarity, dot product, or Euclidean distance.
- Best for standard RAG applications.
- This is what I mainly used in my project.

2. MMR (Max Marginal Relevance)

- It is a retrieval strategy used to fetch relevant and diverse chunks from the vector database.
- The main goal is to avoid retrieving multiple chunks that contain the same information.
- Instead of selecting only the highest-scoring chunks, MMR balances relevance and diversity.
- This helps provide better context to the LLM and improves answer quality.

MMR Lambda

- MMR Lambda controls the balance between relevance and diversity.
- The value ranges between 0 and 1.

- A higher lambda focuses more on relevance.
- A lower lambda focuses more on diversity.
- A balanced value retrieves chunks that are both relevant and non-repetitive.

Real MMR

- If a user asks about KYC requirements, normal similarity search may retrieve multiple chunks explaining the same KYC overview.
- MMR can retrieve chunks covering KYC overview, customer verification, and required documents instead.
- This gives the LLM more complete and diverse context.

3. Hybrid Search

- Combines keyword search and vector search.
- Useful when exact terms are important.
- Common in banking, healthcare, and compliance systems.
- Provides better retrieval for IDs, codes, and policy numbers.

LangChain Chains (LLMChain, SequentialChain, LCEL)

- LangChain is a framework that helps me build LLM applications by connecting components such as prompts, LLMs, retrievers, vector databases, and tools.
- In my project, I used LangChain to build a RAG application where documents were loaded, chunked, converted into embeddings, and stored in Pinecone.
- When a user submitted a query, LangChain handled retrieval of relevant document chunks and passed the context to the LLM for response generation.
- I also used Prompt Templates and Retrievers provided by LangChain to simplify development.
- This reduced the amount of custom code I needed to write and helped standardize the workflow.
- For multi-agent workflows, I used LangGraph on top of LangChain, and I exposed the solution through FastAPI APIs.

LLM Chain

- LLM Chain is the simplest workflow in LangChain.
- It takes user input, combines it with a prompt template, sends it to the LLM, and returns the response.
- It is mainly used for single-step tasks such as summarization, classification, translation, and question answering.
- In my project, I used LLM Chains for tasks where only one LLM call was required without additional retrieval or processing.

Sequential Chain

- Sequential Chain is used when multiple processing steps need to happen in sequence.
- The output of one chain becomes the input for the next chain.

- It helps break complex workflows into smaller logical steps.
- For example, a workflow may first summarize a document, then classify the issue, and finally generate recommendations.
- This approach improves maintainability and allows each step to focus on a specific task.

Components of LangChain

- **Document Loaders** – Used to load data from PDFs, Word files, CSVs, websites, databases, and cloud storage.
- **Text Splitters** – Split large documents into smaller chunks before embedding generation.
- **Embedding Models** – Convert text into vector representations for semantic search.
- **Vector Stores** – Store embeddings and perform similarity search. Examples: Pinecone, FAISS, Chroma.
- **Retrievers** – Retrieve the most relevant document chunks from the vector database.
- **Prompt Templates** – Create dynamic prompts by combining user input and context.
- **LLMs / Chat Models** – Generate responses using models such as GPT-4, Claude Sonnet, or Nova Pro.
- **Chains** – Connect multiple components together into a workflow.
- **Agents** – Allow the LLM to decide which tools or actions to use dynamically.
- **Memory** – Store conversation history and context across interactions.
- **Tools** – External functions, APIs, databases, calculators, search engines, etc., that the LLM can invoke.
- **Output Parsers** – Convert LLM responses into structured formats such as JSON or Python objects.

1. Document Loaders

Document Loaders are used to load data from different sources such as PDFs, Word documents, CSV files, websites, databases, and cloud storage.

They convert raw files into LangChain document objects that contain content and metadata.

This is usually the first step in a RAG pipeline.

2. Text Splitters

Text Splitters divide large documents into smaller chunks before embedding generation.

Chunking is needed because LLMs have context window limitations.

It improves retrieval quality and reduces hallucinations.

Common parameters are chunk size and chunk overlap.

RecursiveCharacterTextSplitter is the most commonly used splitter.

3. Embedding Models

Embedding models convert text into numerical vectors.

These vectors capture semantic meaning.

Similar text produces similar vectors.

Embeddings are used for semantic search and retrieval.

Examples include Titan Embeddings, OpenAI text-embedding-3-small, and Sentence Transformers.

4. Vector Stores

Vector Stores store embeddings and perform similarity search.

When a query arrives, the query is converted into an embedding and compared against stored vectors.

The most similar vectors are retrieved.

Examples are Pinecone, FAISS, Chroma, and OpenSearch.

5. Retrievers

Retrievers fetch relevant chunks from the vector database.

They act as the bridge between the vector database and the LLM.

Instead of sending all documents to the model, only the most relevant chunks are retrieved.

This improves performance and reduces cost.

6. Prompt Templates

Prompt Templates create dynamic prompts.

They combine user input, retrieved context, and instructions.

This ensures consistent prompting across requests.

They also reduce prompt duplication in code.

7. LLMs / Chat Models

LLMs are responsible for reasoning and generating responses.

They receive the prompt and generate the final output.

Examples include GPT-4, Claude Sonnet, and Nova Pro.

The quality of the final response depends heavily on the retrieved context.

8. Chains

Chains connect multiple LangChain components into a workflow.

The output of one component becomes the input of the next component.

Examples include LLMChain and SequentialChain.

Chains simplify workflow management.

9. Agents

Agents allow the LLM to decide which tool or action to use dynamically.

Unlike chains, the execution path is not fixed.

Agents can interact with APIs, databases, search engines, and external tools.

They are commonly used in agentic AI systems.

10. Memory

Memory stores conversation history between interactions.

It helps maintain context across multiple user messages.

Without memory, each request is treated independently.

Examples include ConversationBufferMemory and summary memory.

11. Tools

Tools are external functions that the LLM can invoke.

Examples include APIs, databases, calculators, search engines, and custom Python functions.

Agents typically use tools to perform actions outside the LLM.

12. Output Parsers

Output Parsers convert LLM responses into structured formats.

They help return JSON, dictionaries, or custom objects instead of plain text.

This makes integration with downstream applications easier.

Langgraph

- LangGraph is an orchestration framework built on top of LangChain and is mainly used to build multi-agent and stateful AI workflows.

- In my project, I use LangGraph when a single LLM is not enough and multiple agents need to collaborate to complete a task.
- For example, I have a Retrieval Agent, Validation Agent, Compliance Agent, and Response Agent, and LangGraph manages the flow between them.
- It works using Nodes, Edges, and Shared State, where each node represents an agent or function, edges define the execution path, and state stores information shared across agents.
- It also supports conditional routing, loops, retries, and human-in-the-loop approvals.
- Compared to LangChain chains, LangGraph is better for complex workflows where decisions depend on previous agent outputs.
- In my banking project, I used LangGraph to orchestrate retrieval, compliance validation, and final response generation before sending the answer to the user.

Langchain vs langgraph

- I used LangChain for building the core RAG pipeline, including document loading, chunking, embeddings, retrieval, and prompt management.
- For multi-agent workflows, I used LangGraph on top of LangChain.
- LangGraph allowed me to build agent-based systems with routing, state management, validation, and conditional execution.
- In simple terms, LangChain helps build LLM applications, while LangGraph helps orchestrate complex multi-agent workflows.

Structured Output so-

- Structured Output means forcing the LLM to return data in a predefined format such as JSON instead of free-form text.
- It makes the response consistent and easier for applications to process.
- Structured outputs are commonly used in APIs, automation workflows, databases, and agent-based systems.
- Without structured output, the response format can vary, making integration difficult.
- With structured output, downstream systems can directly parse and use the data.

How I Implemented It

- **Using Prompt Engineering** → Ask the model to return JSON only.
- **Using Pydantic Output Parsers in LangChain.**
- **Using JSON schema validation to ensure the response follows the required format.**

Real Project Use Cases

- Document extraction
- Loan application processing
- KYC data extraction
- Compliance and regulatory workflows
- Agent-to-agent communication

- API responses

LLM + Database Integration

- LLM + Database Integration allows users to query databases using natural language instead of writing SQL.
- The LLM converts the user question into an SQL query, executes it on the database, and returns the result in a user-friendly format.
- This is commonly used in chatbots, reporting systems, BI tools, and analytics applications.

End-to-End Flow

- User asks a question in natural language.
- LLM converts the question into SQL.
- SQL is executed on the database.
- Database returns the result.
- LLM formats the result into a readable response.

Fine Tuning

- Fine tuning means taking an already pre-trained Large Language Model (LLM) and training it again using custom domain data.
- The model already knows language, grammar and general knowledge.
- During fine tuning, we teach the model additional business-specific knowledge.
- Example:
  Before Fine Tuning:
  Model gives generic answer for banking questions.

  After Fine Tuning:
  Model understands banking workflows, KYC rules and compliance-related responses.

LoRA (Low Rank Adaptation)

LoRA is a lightweight fine-tuning technique. Instead of training the full model, LoRA freezes the original model weights and adds small trainable adapter layers.

Only the adapter layers are trained. Original GPT model remains unchanged.

Adapter

Adapter means a small trainable layer added on top of the original LLM. These adapters learn domain-specific behavior while the original model stays frozen.

QLoRA (Quantized LoRA)

QLoRA means Quantized Low Rank Adaptation. It combines LoRA with quantization.

Before training, the original model is compressed into 4-bit format. Then LoRA adapters are attached and trained.

Quantization

Quantization means compressing model weights into lower precision numbers to reduce memory usage.

RAG vs Fine Tuning

RAG (Retrieval-Augmented Generation)

RAG retrieves information from external documents at runtime.

No model retraining is required.

Knowledge can be updated by simply adding new documents.

Best for frequently changing information.

Provides citations and source-based answers.

Lower cost compared to fine-tuning.

Fine-Tuning

Fine-tuning updates the model using custom training data.

Changes the model's behavior and domain understanding.

Requires training infrastructure and GPU resources.

Best for stable domain knowledge and response patterns.

Knowledge updates require retraining.

Used when prompt engineering and RAG are not enough.

Rvf

In my projects, I mainly used RAG because banking and compliance documents changed frequently.

RAG allowed us to update knowledge by simply adding new documents without retraining the model.

I would use fine-tuning when I need to change the model's behavior, response style, or domain understanding.

If the requirement is knowledge retrieval, I prefer RAG. If the requirement is behavior customization, I prefer fine-tuning.

FastAPI

FastAPI is a Python web framework that I used to build REST APIs for GenAI and RAG applications.

It acts as the backend layer between the frontend application and the LLM services.

In my project, user requests came through FastAPI endpoints, and FastAPI handled document retrieval, LLM calls, and response generation.

I preferred FastAPI because it is lightweight, high-performance, supports asynchronous programming, and is easy to integrate with LangChain and AWS services.

FastAPI automatically generates Swagger UI documentation, which helped us test APIs without using Postman.

It also provides request validation using Pydantic models and returns responses in JSON format.

How I Used It

Created REST APIs for chatbot and RAG applications.

Accepted user queries through POST endpoints.

Called LangChain, Pinecone, and LLM services.

Returned generated responses to the frontend.

Deployed the APIs using Uvicorn and Docker on AWS EC2.

What is Streamlit

- Streamlit is a Python framework used to build interactive web applications using only Python code.
- It is mainly used for AI, GenAI, machine learning, data science, and dashboard applications.
- It allows developers to quickly build a user interface without using HTML, CSS, or JavaScript.

How I Used Streamlit

I used Streamlit as the frontend interface for our RAG chatbot for POC

Users entered their questions or uploaded documents through the Streamlit UI.

The request was sent to FastAPI using REST APIs.

FastAPI executed the RAG pipeline by calling LangChain, Pinecone, and Amazon Bedrock.

The generated response was returned to Streamlit and displayed to the user.

Streamlit Components I Used

- **st.title()** – Display application title.
- **st.text_input()** – Accept user questions.
- **st.button()** – Submit user requests.
- **st.file_uploader()** – Upload documents.
- **st.chat_input()** – Chatbot input.
- **st.chat_message()** – Display chat conversation.
- **st.write()** – Display responses.

AWS

- We used AWS Cloud in our project. In that, we used services like S3, IAM, EC2, Lambda, Bedrock, Cloudwatch

Amazon S3

- Amazon S3 is an object storage service used to store files and documents.
- I used S3 to store PDFs, DOCX files, TXT files, images, scanned documents, and knowledge base documents for the RAG application.
- All documents were uploaded to S3 before ingestion and processing.

AWS Lambda

- AWS Lambda is a serverless compute service that runs code without managing servers.

- I used Lambda to automatically trigger the ingestion pipeline when new documents were uploaded to S3.
- Lambda performed preprocessing, validation, and initiated OCR and embedding workflows.

Amazon Bedrock

- Amazon Bedrock is a managed AWS service that provides access to foundation models and LLMs.

- I used Bedrock to access models such as Nova Pro and Claude Sonnet.
- It was used for question answering, summarization, reasoning, and response generation.

Amazon EC2

- Amazon EC2 is a virtual server service used to run applications in the cloud.

- I hosted the FastAPI backend, LangChain application, and RAG services on EC2.
- The application was deployed inside Docker containers running on EC2.

IAM (Identity and Access Management)

- IAM is used to manage authentication, authorization, users, roles, and permissions in AWS.

- I created IAM roles and policies to control access between S3, Lambda, Bedrock, Textract, and EC2.
- It ensured secure access to AWS resources.

Amazon Textract

- Amazon Textract is an OCR service that extracts text, tables, forms, and key-value pairs from documents.

- I used Textract for scanned PDFs and image-based documents where normal text extraction was not possible.
- The extracted text was later used for chunking and embedding generation.

Amazon CloudWatch

- CloudWatch is AWS's monitoring and logging service.

- I used CloudWatch to monitor Lambda executions, application logs, API failures, and system performance.
- It helped with troubleshooting and debugging production issues.

Statistics

Correlation

Correlation means how the two variables are moving together.
Positive correlation means if the both variables are go up or down together
Say for example, Ice cream sales and the hot weather, both will be increasing together
Negative correlation means one variable goes up and the other one will go down,
Say for example, Play time increases and study hours, if play time increases then study hours will decrease.
No Correlation Means No links between those variables,
Show size and the IQ level
Co-efficient

Correlation co-efficient means how much these variables strongly together

It will be a numeric value measured from minus 1 to plus 1 (-1 to +1)

Range: Always between -1.0 (perfect negative) and +1.0 (perfect positive).
Value close to 0: Weak or no linear correlation (e.g., 0.1).
Value close to +1: Strong positive correlation (e.g., 0.9).
Value close to -1: Strong negative correlation (e.g., -0.8).
Answers: "Exactly *how* strong and in *what direction* is that relationship?"
Variance

Variance **measures how far data points spread out from their average (mean) value.**

Median

Median is a middle value of the arranged order dataset.

Regression

Regression is a method to find a best fit of the line or curve through data points to understand how one variable affect the other variable.

Regression is useful to

✓ **Make predictions** – like future sales, stock prices, temperature, etc.
✓ **Understand relationships** – like how salary depends on experience.
✓ **Measure impact** – like how much marketing spend increases sales.
✓ **Forecast trends** – like business growth over time.
✓ **Support decisions** – helps bankers, business owners, doctors, and engineers make better decisions using data.

Machine Learning

Machine Learning is a part of Artificial Intelligence where systems learn patterns from data and improve automatically without writing fixed rules for every condition.

For example:

- In banking, if we give past transaction data to the model, it can learn which transactions are fraud and which are normal.
- In healthcare, it can learn from patient records and predict disease risk.
- In e-commerce, it can recommend products based on customer behavior.

Normal programming:

- We write rules manually.
- Input + Rules → Output

Machine Learning:

- We give data and expected results.
- Input + Output data → Model learns rules automatically

Types of Machine Learning:

1. Supervised Learning

    - Has input and output labels
    - Example: spam detection, fraud detection
2. Unsupervised Learning
    - No labels
    - Finds hidden patterns or groups
    - Example: customer segmentation
3. Reinforcement Learning
    - Learns using rewards and penalties
    - Example: robotics, game AI

ML Data types

Category	Type	Description	Example	Math Operations	Graph
Qualitative	Nominal	Names only	Gender, City	✕ None	Bar / Pie
Qualitative	Ordinal	Ordered categories	Rating, Education	− Rank, Median	Ordered Bar
Quantitative	Interval	Equal gaps, no true zero	Temperature (°C), Year	+ − Only	Histogram / Line
Quantitative	Ratio	Equal gaps, true zero	Age, Salary, Weight	+ − × ÷	Histogram / Scatter

Supervised Learning

1. Supervised learning means the model learns from labeled data where both input and output are available.
2. Here, the model learns the mapping between features (X) and target (y).
3. I use supervised learning when the business already has a target column like fraud flag, churn status, or loan default.
4. Classification is used when predicting categories like Yes/No or Fraud/Non-Fraud.
5. Regression is used when predicting continuous values like sales amount or revenue.
6. Common supervised algorithms are Logistic Regression, Random Forest, XGBoost, SVM, and Neural Networks.
7. We evaluate supervised models using Accuracy, Precision, Recall, F1-Score, ROC-AUC, RMSE, or MAE.
8. Example: age=42, income=80k, tenure=2 years → default=Yes.

Unsupervised Learning

1. Unsupervised learning means the model learns only from input data without any target column.
2. The goal is to discover hidden patterns, clusters, relationships, or anomalies in the data.
3. I use unsupervised learning for customer segmentation, anomaly detection, and exploratory analysis.

4. Clustering is used to group similar records together.
5. Dimensionality reduction is used to reduce feature size and improve visualization or performance.
6. Anomaly detection helps identify unusual transactions or abnormal behavior.
7. Common unsupervised algorithms are K-Means, DBSCAN, PCA, Isolation Forest, and Apriori.
8. We evaluate unsupervised models using Silhouette Score, cluster quality, and business interpretability.
9. Example: age=42, income=80k, tenure=2 years → no target column available.

Reinforcement Learning

1. Reinforcement Learning is a trial-and-error learning method where the model learns from rewards and penalties.
2. The model interacts with an environment and improves its decisions over time.
3. I use Reinforcement Learning when the system needs continuous learning and dynamic decision-making.
4. Main components are Agent, Environment, State, Action, and Reward.

Results After Reinforcement Learning

1. Fraud detection accuracy improved significantly.
2. False positives reduced, so customer complaints became less.
3. The model adapted automatically to new fraud patterns.
4. RL performed better because it learned continuously from real-time feedback.

Supervised Learning → Learn from *labeled data*
Unsupervised Learning → Find *patterns*
Reinforcement Learning → Learn from *experience (reward/punishment)*

Regression

- Regression is a Machine Learning and statistical technique used to predict numerical values based on previous data.
- It helps find the relationship between input variables and output variables.
- I use regression when the output is a continuous number like sales amount, house price, or revenue.
- Example: Predicting house price using size, location, and number of rooms.

Why is Regression Needed?

1. Regression helps make future predictions using historical data.
2. It helps understand relationships between variables, like salary based on experience.
3. It measures the impact of one variable on another, like marketing spend on sales.
4. It helps forecast trends such as business growth, stock prices, or demand prediction.

5. It supports data-driven decision-making in banking, healthcare, retail, and other domains.

Without Regression, What Will Happen?

1. We cannot predict future numerical values accurately.
2. Business decisions will depend more on guesswork instead of data.
3. It becomes difficult to identify relationships between variables.
4. Trends and patterns in the data may be missed.
5. Forecasting and planning become less effective.

Regression Types

- **Simple Linear Regression**
- **Multiple Linear Regression**
- **Polynomial Regression**
- **Ridge Regression (L2 Regularization)**
- **Lasso Regression (L1 Regularization)**
- **Elastic Net Regression**
- **Logistic Regression**
- Quantile Regression
- Support Vector Regression (SVR)
- **Decision Tree Regression**
- **Random Forest Regression**
- **XGBoost Regression**
- Bayesian Regression
- Poisson Regression
- Robust Regression

Linear Regression

- Linear Regression is a supervised machine learning algorithm used to predict numerical values.
- It finds the relationship between independent variables and dependent variables.
- It assumes a linear relationship, where a change in input causes a proportional change in output.
- Example: Predicting house price, sales amount, or student marks.
- It has two types simple linear and multi linear regression.

Simple Linear Regression

- It Uses one independent variable and one dependent variable.
- Example: Predicting exam score based on study hours.

Multiple Linear Regression

- It Uses multiple independent variables and one dependent variable.
- Example: Predicting house price using size, location, and number of bedrooms.

Polynomial Regression polynomial

- Polynomial Regression is a type of regression where the relationship between X and Y is curved instead of a straight line.
- It is an extended version of Linear Regression.
- I use Polynomial Regression when data shows non-linear patterns.
- Example: Age vs Income, Speed vs Mileage, Temperature vs Performance.
- In Age vs Income, income increases up to a certain age and then decreases, so the relationship becomes curved.
- Linear Regression cannot fit this pattern properly because it draws only a straight line.
- Polynomial Regression fits a curved line and captures the pattern better.

Overfitting in Polynomial Regression

- Overfitting happens when the polynomial degree becomes too high.
- The model starts fitting every data point, including noise and random fluctuations.
- It performs well on training data but fails on new or unseen data.

Degree Behavior

- Degree 1 → Straight line → Underfitting
- Degree 2 or 3 → Smooth curve → Good fit
- Very high degree → Wiggly curve → Overfitting

How to Avoid Overfitting

- We Use Regularization techniques like Ridge or Lasso.
- Use Cross-Validation to test on unseen data.
- Choose the optimal polynomial degree instead of increasing it blindly.
- Visualize the curve; if it looks too wiggly, it is likely overfitting.

Regularization

- Regularization is a technique used to reduce overfitting in regression models.
- It adds a penalty to large coefficients and makes the model simpler and more stable.
- We used ridge and lasso

Ridge Regression (L2 Regularization)

- Ridge Regression adds L2 penalty, which is the sum of squares of coefficients.

- It reduces coefficient values but does not make them exactly zero.
- I use Ridge when there are many correlated features and I want to keep all features in the model.
- It helps reduce overfitting and improves model generalization.

Lasso Regression (L1 Regularization)

- Lasso Regression adds L1 penalty, which is the sum of absolute values of coefficients.
- It can reduce some coefficients exactly to zero.
- I use Lasso when I want feature selection and a simpler interpretable model.
- It automatically removes less important features and reduces overfitting.

Elastic Net Regression

- Elastic Net Regression is a regularized linear regression technique that combines both Ridge (L2) and Lasso (L1) regularization.
- It gives the benefits of both Ridge and Lasso together.
- Ridge helps reduce large coefficients, while Lasso helps remove unimportant features.
- Elastic Net both shrinks coefficients and can set some coefficients to zero.

When to Use Elastic Net

- I use Elastic Net when there are many correlated or noisy features.
- It is useful when we need both feature selection and model stability.
- It works well for high-dimensional datasets where the number of features is large.

Logistic Regression

- Logistic Regression is a supervised machine learning algorithm used for classification problems.
- It is mainly used for binary classification like Yes/No, True/False, 0/1 predictions.
- It predicts the probability of an event occurring using a Sigmoid (S-shaped) function.
- If probability is greater than or equal to 0.5, it predicts Class 1; otherwise, it predicts Class 0.

When to Use Logistic Regression

- I use Logistic Regression when the output variable is categorical.
- Common use cases are loan default prediction, fraud detection, disease prediction, customer churn, and employee attrition.
- It works best when there is a linear relationship between features and log-odds.

Key Features

- Output is always a probability between 0 and 1.

- It uses the Sigmoid activation function.
- It is simple, fast, and easily interpretable.
- Coefficients show how features influence the prediction outcome.

Regularization in Logistic Regression

- L1 Regularization (Lasso) removes less important features by making coefficients zero.
- L2 Regularization (Ridge) reduces large coefficients and helps prevent overfitting.
- Regularization improves model stability and generalization on unseen data.

Quantile Regression

- Quantile Regression is a regression technique used to predict different quantiles or percentiles of the target variable instead of only the average value.
- In normal Linear Regression, we predict the mean of the output, but Quantile Regression predicts values like 25th, 50th, and 75th percentiles.
- I use Quantile Regression when the data has high variation, outliers, or when I want to understand low, medium, and high-level outcomes separately.

When to Use Quantile Regression

- When prices, income, or losses vary widely.
- When the dataset contains outliers.
- When percentile-based predictions are needed.
- When feature impact changes across low-end and high-end values.

Support Vector Regression (SVR)

- Support Vector Regression (SVR) is a regression algorithm that predicts values within a margin of tolerance called the ε-tube.
- Small errors inside the margin are ignored, while larger errors are penalized heavily.
- Unlike Linear Regression, SVR focuses mainly on minimizing major errors instead of all errors.
- I use SVR when the relationship between input and output is non-linear and the data contains noise or fluctuations.

Decision Tree Regression

- Decision Tree Regression is a tree-based regression algorithm that splits data into smaller groups based on feature conditions.
- Each split tries to make the target values in that region as similar as possible.
- The final prediction is the average value of the target variable in the leaf node.
- It works like a flowchart or if-else rule system.

Random Forest Regression

- Random Forest Regression is an ensemble machine learning algorithm that combines multiple Decision Trees to make predictions.
- Each tree is trained on random subsets of data and features, so every tree learns slightly different patterns.
- Final prediction is the average of all tree predictions, which improves accuracy and stability.
- In simple terms, Random Forest means many Decision Trees working together to make one strong prediction.
- It helps reduce overfitting and variance compared to a single Decision Tree.

XGBoost Regression

- XGBoost stands for Extreme Gradient Boosting and is an advanced ensemble regression algorithm.
- It builds multiple Decision Trees sequentially, where each new tree learns from the errors of previous trees.
- This process is called boosting.
- XGBoost uses gradient descent optimization to reduce prediction error step by step.
- In simple terms, each tree corrects the mistakes of earlier trees, Bayesian Regression Bayesian- bay-

- Bayesian Regression is an advanced version of Linear Regression that predicts values along with uncertainty or confidence intervals.
- Instead of fixed coefficients, it treats coefficients as probability distributions using Bayes' Theorem.
- Instead of giving one exact prediction, it provides a range of likely predictions with confidence levels.
- For example, instead of saying "House price is ₹75L," it says "House price is around ₹75L ± ₹5L with 95% confidence."

When to Use Bayesian Regression

- When uncertainty estimation is important.
- When the dataset is small or noisy.
- When probabilistic predictions are needed.
- When avoiding overfitting is important.
- Common use cases are finance risk modeling, healthcare predictions, and time-series forecasting.

Poisson Regression

- Poisson Regression is a regression algorithm used for count-based target variables like number of calls, visits, bookings, or transactions.
- It assumes the target variable follows a Poisson distribution.
- The output values are always non-negative integers like 0, 1, 2, 3, and so on.
- It predicts event occurrence rates instead of continuous values.

Robust Regression

- Robust Regression is a regression technique used to handle outliers and noisy data.
- Unlike normal Linear Regression, it reduces the influence of extreme values on the prediction line.
- In Linear Regression, even one outlier can distort the regression line significantly.
- Robust Regression uses different loss functions to reduce the impact of large errors.

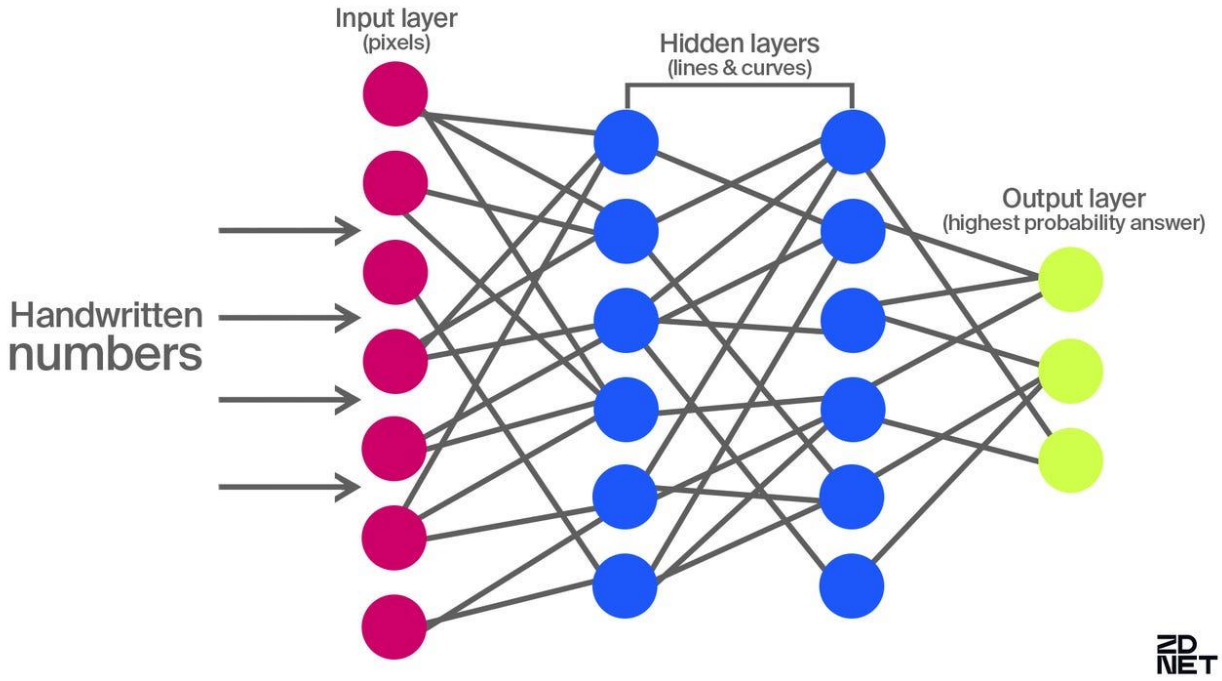
No	Regression Type	When to Use	Pros ✓	Cons ✗	When Not to Use ⊘
1	**Linear Regression**	When the relationship between X and Y is roughly **straight-line**	Easy to understand & fast	Fails with outliers or curved data	If data is **non-linear** or has **outliers**
2	**Polynomial Regression**	When the data shows a **curved pattern** (e.g., U-shape, exponential rise)	Captures curves easily	Can **overfit** if degree too high	If trend is linear or you have little data
3	**Quantile Regression**	When you want to study **low, median, or high** parts of target variable	Robust to outliers; detailed insights	Harder to interpret	If only the **average trend** matters
4	**Support Vector Regression (SVR)**	When the relation is **non-linear** but you want to ignore small noise	Handles complex trends; ignores small errors	Complex tuning (kernel, C, ε); slower	For **very large datasets** or purely linear data
5	**Decision Tree Regression**	When you want **if–else rule-based predictions**	Easy to interpret; handles non-linearity	Can **overfit** small data; unstable	If you need smooth or continuous trends
6	**Random Forest Regression**	When you want **high accuracy** and **non-linear pattern handling**	Reduces overfitting; handles missing data	Slower; less interpretable	If data is **small/simple** (use Linear/Tree)

No	Regression Type	When to Use	Pros ✓	Cons ✗	When Not to Use ⊘
7	**XGBoost Regression**	When data is **large, complex, and non-linear**	Very accurate; learns from mistakes; handles noise	Needs tuning; less interpretable	For **tiny datasets** or purely linear data
8	**Bayesian Regression**	When you want **uncertainty estimates** with predictions	Gives confidence range; robust on small data	Computationally heavy; complex math	When you just need a quick average fit
9	**Poisson Regression**	When target is **count-based** (e.g., number of events)	Ensures positive predictions; fits count data	Assumes mean ≈ variance; can't handle negatives	When target is **continuous** or not count-based
10	**Robust Regression**	When data has **outliers or bad values**	Ignores outliers automatically; stable fit	Slightly less accurate on clean data	When data is already **clean and consistent**

Key Points

Regression Type	Predicts	When to Use	Key Feature
Logistic	Class (0/1)	Binary classification	Uses sigmoid
Quantile	Percentiles	Analyze full distribution	Handles skewed data
SVR	Continuous	Non-linear data	Margin-based learning
Decision Tree	Continuous	Simple, rule-based	Interpretable
Random Forest	Continuous	Large, complex data	Averaged trees
XGBoost	Continuous	High accuracy, big data	Sequential boosting

Regression Type	Predicts	When to Use	Key Feature
Bayesian	Continuous	Uncertainty needed	Probabilistic
Poisson	Count	Event frequency	For count outcomes
Robust	Continuous	Outliers present	Ignores extreme values



Deep Learning

• Deep Learning is a subset of Machine Learning that uses Neural Networks with multiple hidden layers to learn complex patterns from data.
• It is inspired by the human brain, where neurons work together to process information.
• It is mainly used in NLP, computer vision, speech recognition, recommendation systems, and Generative AI.

How Deep Learning Works

1. First, data enters through the Input Layer.
2. Then Hidden Layers process the data using weights, bias, and activation functions.
3. During Forward Propagation, predictions are generated.
4. Loss Function calculates prediction error.
5. Backpropagation updates the weights to reduce error.

6. Optimizers like Adam or SGD improve the model accuracy.
7. This process repeats multiple times until the model learns patterns properly.

Neural Networks

Input Layer
• Input layer is the first layer in a Neural Network.
• It receives raw input data like text, images, audio, or numbers and sends it to hidden layers for processing.

Hidden Layers
• Hidden layers are intermediate layers between input and output layers.
• They learn patterns, relationships, and important features from the data automatically.

Output Layer
• Output layer gives the final prediction or result of the model.
• The number of neurons depends on the problem type like classification or regression.

Perceptron
• Perceptron is the basic building block of a Neural Network.
• It takes inputs, applies weights and bias, then produces an output using an activation function.

Activation Functions
• Activation functions decide whether a neuron should activate or not.
• They help Neural Networks learn complex and non-linear patterns from data.

Weights and Bias
• Weights determine the importance of each input feature in prediction.
• Bias helps shift the output and improves model learning flexibility.

Forward Propagation
• In forward propagation, data moves from input layer to output layer step by step.
• The model generates predictions using current weights and biases.

Backpropagation
• Backpropagation calculates prediction error and sends it backward through the network.
• It updates weights to reduce loss and improve model accuracy.

Loss Function
• Loss function measures how far the predicted output is from the actual value.
• The main goal during training is to minimize this loss.

Optimizers
• Optimizers help update weights efficiently during training.
• Common optimizers like Adam and SGD improve convergence and model performance.

Activation Functions

ReLU
• ReLU stands for Rectified Linear Unit and is the most commonly used activation function.
• It converts negative values to 0 and keeps positive values unchanged for faster training.

Sigmoid
• Sigmoid gives output values between 0 and 1.
• It is mainly used in binary classification problems.

Tanh
• Tanh gives output values between -1 and 1.
• It is better centered around zero compared to sigmoid.

Softmax
• Softmax is mainly used for multi-class classification problems.
• It converts outputs into probability values whose total becomes 1.

Activation Functions Importance
• Activation functions help Neural Networks learn non-linear and complex relationships.
• Without activation functions, Neural Networks behave like simple linear models.

Python

- ✓ In python, variable are used to store values and we do not need to declare the variable as we do in other scripting languages.
- ✓ Yeah amusing pycharm
- ✓ Some common data types that we use in python are **integers, floats, strings, Booleans, lists, tuples** and **dictionaries**.
- ✓ In this, Lists are mutable sequences, tuples are immutable sequences, and dictionaries are key-value pairs.
- ✓ In addition, it supports all the arithmetic, comparison and logical operators.
- ✓ We can also use conditional statement like **IF, ELIF, ELSE**, and loops like **FOR, WHILE**.
- ✓ Moreover, functions are defined using the **DEF** keyword.
- ✓ In addition to that we can use some standard library using the **IMPORT** statement.

List

- Lists are mutable
- Means elements of a list can be changed, added, or removed after the list is created.
- Lists are defined using square brackets `[]`
- Since lists are mutable, they generally have a slow in performance when compared to Tuple
- Generally Its suitable for collections of items that may change over time
- My_list = [1,2,3,4]
- Array requies a fixed type and are

Tuple

- Tuples are immutable
- Means once a tuple is created, its elements cannot be changed, added, or removed.
- Tuples are defined using parentheses `()`
- Generally Its suitable for fixed collections of items.
- My_tuple = (1,2,3,4)

Set

- It's the collection of unordered and unique items
- Sets are defined using curly braces `{}`
- Sets are mutable

Dictonary

- Dictionary is an unordered collection of key-value pairs.
- Dictionaries are defined using curly braces `{}` with key-value pairs separated by colons `:`
- Dictionaries are mutable
- Keys in a dictionary must be unique, but values can be duplicated.

Array

- Arrays can only store elements of the same data type
- Arrays are more memory-efficient than lists

Series

- **Series** is a one-dimensional array
- It can have any data of any type (integers, strings, floats, etc.).
- Series consists of an index and values

Dataframe

- **DataFrame** is a two-dimensional
- tabular data structure with rows and columns
- its used to store the data from the table in the form of dataframe
- inside the dataframe variable

Lamda

- It can take any number of arguments but only has a single expression.
- Lambda functions are syntactically restricted to a single line and are often used for short, simple operations.
- Its mostly used with functions like `sorted()`, `filter()`, and `map()` where a function is required as an argument.

- Example :
    - multiply = lambda a, b: a * b
    - print(multiply(3, 4))

Functions

- Function is defined using the `def` keyword
- It can be defined outside of a class and can be called independently
- Functions can be called directly using their name and passing the required arguments
- Can access global variables

Methods

- Method is a function that is defined within a class and is associated with instances of that class.
- Methods can be called on an instance of the class
- First parameter is usually `self`
- Its associated with a class and its instances
- Methods are called on objects (instances of a class) using the dot notation.
- Can access instance and class attributes

Break

- The `break` statement is used to terminate the loop prematurely
- When `break` is encountered inside a loop like for or while,
  The loop stops executing, and control flow resumes immediately after the loop.
    - for i in range(10):
        - if i == 5:
            - break
        - print(i)

Output : 0 1 2 3 4

Continue

- The `continue` statement is used to skip the rest of the code inside the current iteration of the loop and move to the next iteration.
- It does not terminate the loop but skips the current iteration
    - for i in range(10):
        - if i % 2 == 0:
            - continue
        - print(i)

Output: 1 3 5 7 9

Pass

- it is used as a placeholder in loops

- When `pass` is executed, nothing happens
    - for i in range(10):
        - if i < 5:
            - pass
        - else:
            - print(i)

Output: 5 6 7 8 9

Strip- to remove the leading and trailing spaces

Upper- to convert the string to upper case , txt = txt.upper()

Generator

- Generators is a type of iterator.
- Its simpler to create and more memory-efficient.
- Instead of creating the entire sequence in memory at once. It will iterate over a sequence of values lazily, one value at a time.
- Since values are generated one by one, generators are very memory-efficient, especially with large datasets.
- Generators are created using functions that include the `yield` statement instead of `return`.

Decorator

Decorator is a function that takes another function as an argument, adds some functionality, and returns a new function.

Iterators

An iterator in Python is an object that allows you to traverse through all the elements of a collection (like lists, tuples, or dictionaries) one element at a time. Iterators implement two methods: `__iter__()` and `__next__()`.

The `__iter__()` method returns the iterator object itself and is implicitly called at the start of loops.

The `__next__()` method returns the next element from the collection and raises a `StopIteration` exception when there are no more elements.

Slicing

- **Slicing** is the process that is used to extract a portion of a sequence, such as a list, tuple, string.
- Syntax for slicing is **Sequence[Start:stop:step]**

`split()` function is a built-in method used to split a string into a list of substrings based on a specified separator.

Range vs XRange

- **Both range and xrange() functions are similar to each other**
- **Its used to generate a sequence of numbers.**
- **But xrange() is removed from Python 3. whereas the range() function is used in Python 3**

Merge

- merged_df = pd.merge(df1,df2, how='outer', on = 'deptid',indicator = True)
- difference = merged_df[merged_df['_merge'] 1= 'both']
- how = 'outer' means the outer join, it means it will form a single dataframe by merging both df1 and df2. It will have data which are matching between two dataframes as well as the unmatched data between the dataframes.
- Indicator True, This will adds a special column '_merge' to the merged_df. This column indicated the source of each row. And the possible values are 'Left_only','right_only','both'

Group

- Grouped = merged_df.groupby('Deptname')
  Sum_of_salary = Grouped['Salary].sum()